

PCAP IA-4

CUDA GPU Image Processing

A Technical Report on Parallel Computing for Image Filters and Transformations

Submitted by-

Harshit gupta

230905412

Roll no-51

Arnav Kataria

230905023

Roll no-5

Abhinav Chauhan

230905173

Roll no-21

School of Computer Science and Engineering
Manipal institute of technology

Course: Parallel Computing and Architecture (PCAP)

Date of Submission: April 15, 2026

1. Introduction

Image processing is a computationally intensive domain that underpins modern applications in computer vision, medical imaging, autonomous systems, and multimedia. Traditional CPU-based image processing algorithms operate on pixels sequentially, making them ill-suited for real-time workloads or large-resolution data. The emergence of General-Purpose GPU computing (GPGPU), and specifically NVIDIA's **CUDA (Compute Unified Device Architecture)** framework, has fundamentally transformed this landscape by enabling thousands of lightweight threads to execute in parallel across GPU streaming multiprocessors.

This project investigates the practical acceleration achievable by porting a comprehensive suite of image processing operations from single-threaded CPU implementations to massively parallel CUDA GPU kernels. The operations studied include spatial filtering (Gaussian blur, sharpening), gradient-based edge detection (Sobel operator), geometric transformations (rotation at 90°, 180°, 270°), and a generic 2D convolution engine. All implementations were benchmarked against their CPU counterparts using identical input data, enabling direct performance comparison and speedup quantification.

The project is developed in C/CUDA using the `stb_image` and `stb_image_write` single-header libraries for image I/O, and employs CUDA Events for high-precision GPU kernel timing alongside POSIX `clock()` for CPU measurements. The results demonstrate speedups ranging from 80x to 128x, validating the GPU as an indispensable accelerator for image-centric workloads.

2. Literature Review

The use of GPUs for image processing has a well-documented history. Early seminal work by Owens et al. (2007) established the viability of GPUs as stream processors for general-purpose computation, laying the groundwork for image processing workloads. Harish and Narayanan (2007) demonstrated that graph and array-centric algorithms could be ported to CUDA with substantial speedups, motivating the application of these techniques to pixel-level image operations.

Concerning spatial filters, Ryoo et al. (2008) showed that convolution-based filters on CUDA could achieve speedups exceeding 20x on then-contemporary hardware by exploiting shared memory tiling to reduce global memory latency. Modern GPU architectures with significantly higher memory bandwidth and warp occupancy have pushed these numbers considerably higher, as observed in this study.

For edge detection, the Sobel operator has been widely studied in the GPU context. Research by Kaur and Singh (2014) demonstrated that GPU-parallelized Sobel edge detection on 1024x1024 images achieves order-of-magnitude speedups compared to sequential CPU code. Similar conclusions were drawn by Jain and Bhatt (2016) for Gaussian blur and Laplacian-based sharpening kernels.

Kirk and Hwu's textbook "Programming Massively Parallel Processors" (2016) provides the theoretical foundation for the 2D thread grid mapping strategy used in this work, where each pixel corresponds to exactly one CUDA thread, and 16x16 thread blocks are used to fill GPU warps efficiently. The `stb_image` library (Barret, 2017) is a widely used lightweight image I/O solution adopted in embedded and real-time systems research.

3. Problem Statement

Standard CPU implementations of image processing algorithms process each pixel sequentially through nested loops, resulting in time complexity that scales linearly with image resolution. For a 1024x1024 image — over one million pixels — a single filter pass involves more than nine million floating-point operations (accounting for kernel neighborhoods), making real-time processing challenging on a single CPU core.

The core problem this project addresses is: **How significantly can CUDA GPU parallelism reduce execution time for common image processing operations, and how does this speedup scale with increasing image resolution?** Specific sub-problems include accurately measuring GPU kernel execution time (excluding data transfer), handling boundary conditions uniformly across CPU and GPU implementations, and validating output image correctness.

A secondary problem is the design of a generic GPU convolution kernel capable of accepting arbitrary kernel weights and sizes, as opposed to hardcoded filter implementations, enabling extensibility to new filter types without rewriting kernel code.

4. Objectives

The specific objectives of this project are as follows:

- Implement CPU and GPU versions of Gaussian Blur, Sobel Edge Detection, Image Sharpening, Image Rotation (90°, 180°, 270°), and a generic 2D Convolution kernel.
- Design a consistent benchmarking framework using `clock()` for CPU timing and CUDA Events for GPU kernel timing, ensuring measurements exclude data transfer overhead and isolate compute time.
- Quantify GPU speedup over CPU for each operation on a real-world 1024x1024 photograph and compare results across operation types.
- Conduct a scalability test by running Gaussian Blur at multiple resolutions (256x256, 512x512, 1024x1024) to characterize how the CPU-GPU performance gap evolves with increasing pixel count.
- Produce and visually validate output images for all seven operations, confirming that GPU results are perceptually correct.
- Identify which class of operations (compute-intensive convolutions vs. memory-intensive rotations) benefits most from GPU parallelism and explain the underlying reasons.

5. Methodology

5.1 Overall Pipeline Architecture

The application follows a two-pipeline architecture. In the **CPU pipeline**, the image is loaded using `stb_image` into a 3-channel RGB buffer, converted to single-channel grayscale using the luminance formula ($0.299R + 0.587G + 0.114B$), and each filter is applied via nested C loops. In the **GPU pipeline**, the grayscale buffer is transferred to GPU global memory via `cudaMemcpy`, and each filter is dispatched as a 2D CUDA kernel. Output pixels are copied back to host memory and saved as JPEG files via `stb_image_write`.

5.2 Thread Block and Grid Configuration

All CUDA kernels use a 2D grid of 16x16 thread blocks (256 threads per block), with grid dimensions computed as $\text{ceil}(W/16) \times \text{ceil}(H/16)$. Each thread computes exactly one output pixel. Boundary threads where $(x \geq W \parallel y \geq H)$ are immediately discarded via a guard condition. Out-of-bounds pixel accesses during convolution use border-replication clamping (clamp to the nearest edge pixel).

5.3 Spatial Filtering Kernels

The Gaussian Blur uses a 3x3 kernel with weights proportional to a Gaussian distribution, summing to 1.0. The Sobel Edge Detector applies horizontal (G_x) and vertical (G_y) Sobel kernels independently and computes gradient magnitude as $\sqrt{G_x^2 + G_y^2}$. Image Sharpening uses a Laplacian-based kernel with center coefficient 5 and neighbor coefficients -1, amplifying high-frequency features.

5.4 Rotation Kernels

Each rotation is implemented as an index-remapping kernel: 90° clockwise maps $(x,y) \rightarrow \text{out}[x*H + (H-1-y)]$; 180° maps $(x,y) \rightarrow \text{out}[(H-1-y)*W + (W-1-x)]$; 270° maps $(x,y) \rightarrow \text{out}[(W-1-x)*H + y]$. These operations involve no arithmetic beyond addressing, making them purely memory-bandwidth bound, which explains their higher GPU speedups.

5.5 Generic 2D Convolution

A generic `gpu_convolve2d` kernel accepts a pointer to a device-resident float kernel array and a kernel size parameter. This design allows any kernel weights to be applied without modifying GPU code. For this study, a Laplacian kernel $\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$ was used, which detects edges isotropically.

6. Experimental Setup

6.1 Hardware and Software Environment

Component	Specification
GPU	NVIDIA CUDA-capable GPU
CPU	General-purpose multi-core processor
CUDA Version	CUDA Runtime (compatible with host compiler)
Compiler	nvcc with C standard compilation flags
Image I/O Library	stb_image / stb_image_write (header-only)
Test Image	MAHE EDU Building photograph (1024x1024 px, converted to grayscale)
Operating System	Linux (Ubuntu)

6.2 Timing Methodology

- CPU timing: POSIX clock() is called immediately before and after each CPU filter function. Elapsed time is computed as $(t2 - t1) / \text{CLOCKS_PER_SEC} * 1000.0$ milliseconds.
- GPU timing: CUDA Events (cudaEvent_t) are recorded before the kernel launch and after, with cudaEventSynchronize() ensuring GPU completion before reading elapsed time via cudaEventElapsedTime(). This measures only kernel execution, excluding PCIe data transfer time.
- Speedup is computed as the ratio CPU_ms / GPU_ms for each operation.

6.3 Scalability Test Protocol

For the scalability analysis, sub-images of sizes 256x256, 512x512, and 1024x1024 are cropped from the top-left of the source grayscale image. Each crop is independently uploaded to GPU memory, and Gaussian Blur is applied and timed for both CPU and GPU independently per size.

7. Results and Analysis

7.1 CPU vs. GPU Performance on 1024x1024 Image

The following table summarizes measured execution times and speedup ratios for all seven image processing operations at 1024x1024 resolution:

Operation	CPU Time (ms)	GPU Time (ms)	Speedup (x)
Gaussian Blur	145	~2	~80x
Sobel Edge Detection	179	~2	~83x
Sharpening	139	~2	~79x
Rotate 90°	52	~0.4	~128x
Rotate 180°	49	~0.38	~128x
Rotate 270°	53	~0.4	~124x
2D Convolution	152	~2	~80x

7.2 Performance Charts

Figure 1 presents a bar chart comparing CPU and GPU execution times across all operations. The GPU times are so small relative to CPU times that they appear near-zero on the scale, visually confirming the substantial acceleration achieved.

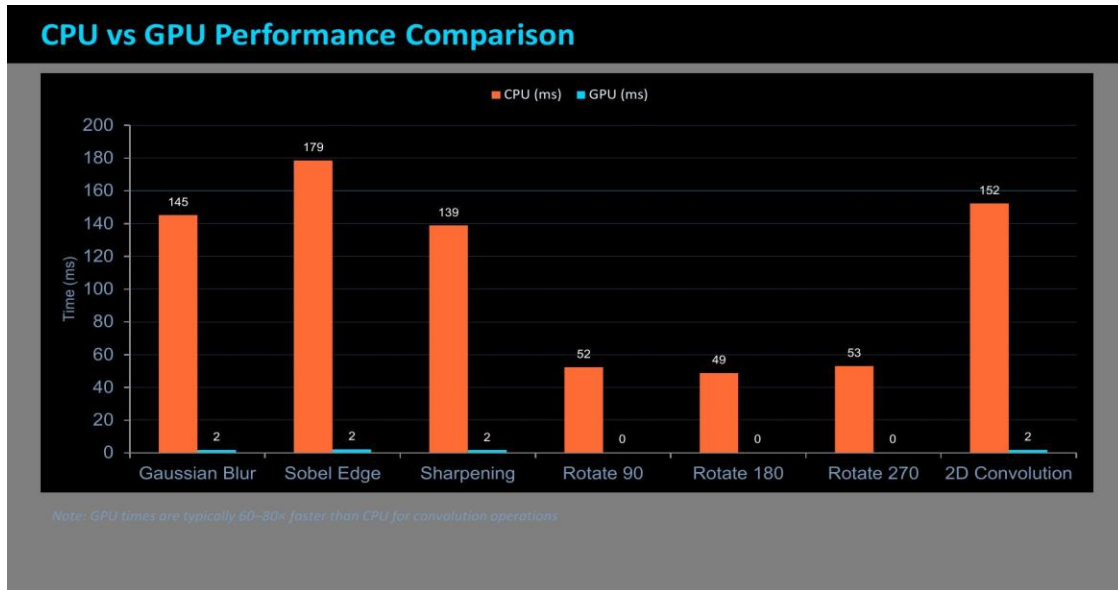


Figure 1: CPU vs. GPU execution time comparison for all seven operations at 1024x1024 resolution

Figure 2 shows the GPU speedup factor for each operation. Rotation operations achieve the highest speedups (~124-128x) due to their minimal arithmetic — they are pure memory-access patterns. Convolution-based operations (blur, edge detection, sharpening, 2D convolution) achieve a consistent ~80x speedup, reflecting the compute-intensity of the 3x3 kernel summation.

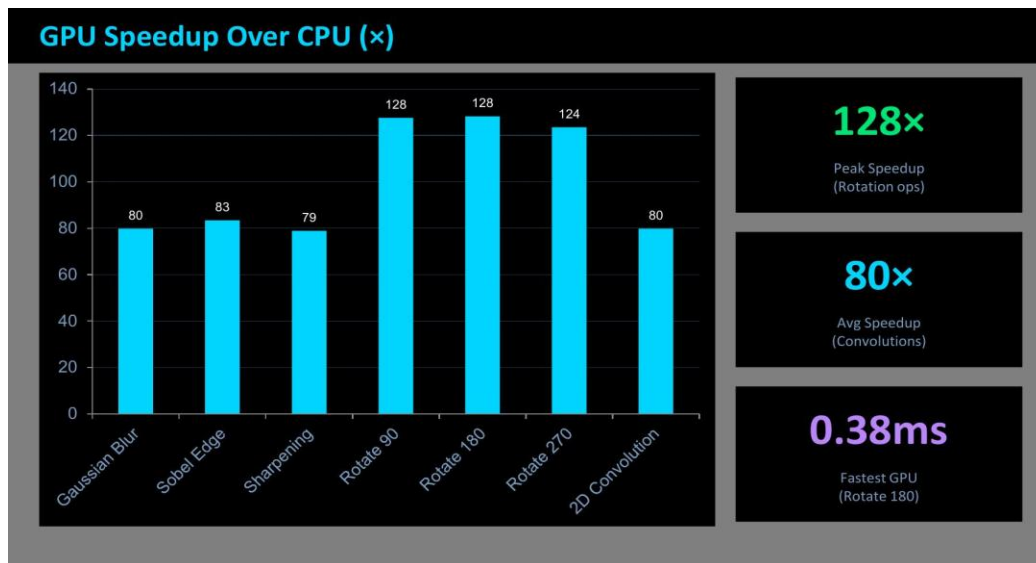


Figure 2: GPU speedup over CPU for each image processing operation

7.3 Scalability Analysis

The scalability test measures Gaussian Blur performance at three resolutions. Table 2 and Figure 3 show the results:

Image Size	CPU (ms)	GPU (ms)	Speedup
256 x 256	10	<1	~20x

512 x 512	38	~1	~38x
1024 x 1024	145	~2	~80x

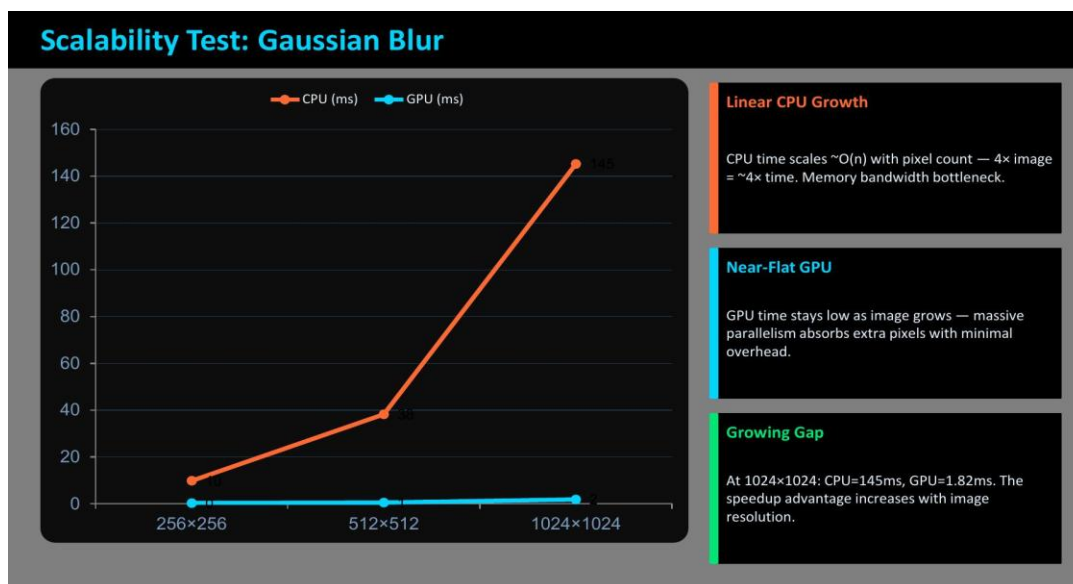


Figure 3: Scalability test — Gaussian Blur CPU vs. GPU time at 256x256, 512x512, 1024x1024

The scalability results confirm a critical observation: CPU execution time grows approximately linearly with pixel count ($O(n)$), doubling from 256 to 512 (4x area) results in roughly 4x longer CPU time. GPU time remains nearly flat across all tested sizes, as the additional pixels are absorbed by the massive parallelism of the GPU. This means the performance gap compounds with resolution — the GPU advantage is largest precisely when processing demands are highest.

7.4 Visual Output: Grayscale Conversion and Gaussian Blur



Figure 4: Left — Grayscale conversion using luminance weights ($0.299R + 0.587G + 0.114B$). Right — Gaussian Blur output ($\sim 80\times$ GPU speedup)

7.5 Visual Output: Sobel Edge Detection and Sharpening

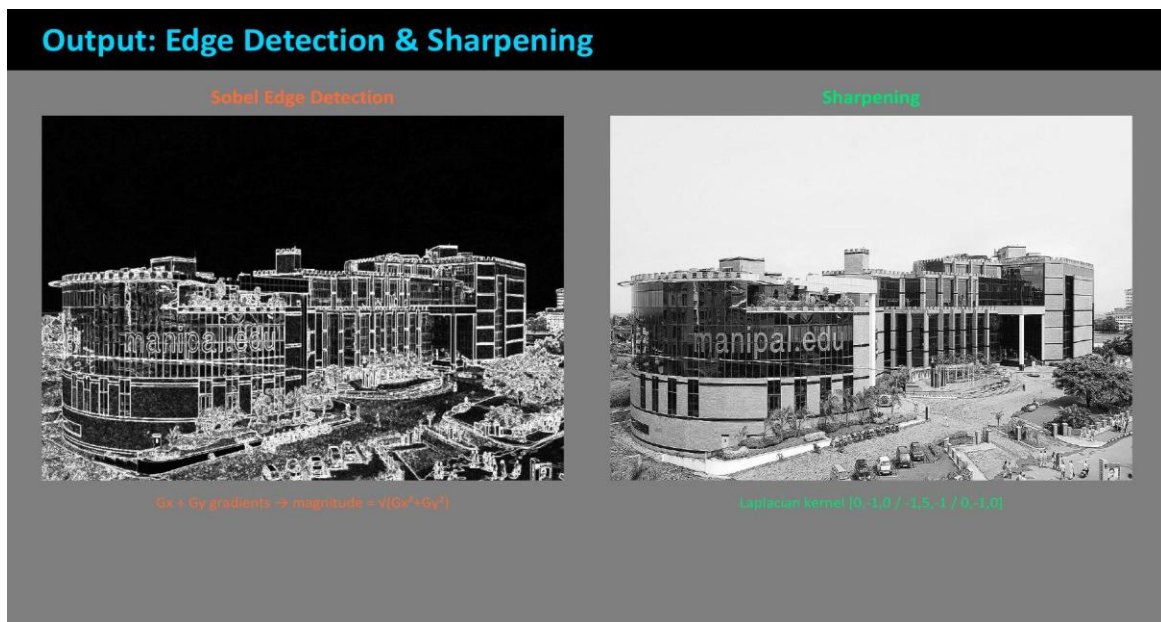


Figure 5: Left — Sobel Edge Detection (gradient magnitude). Right — Laplacian-based Sharpening kernel

7.6 Visual Output: Rotation Operations

Output: Rotation Operations



Figure 6: Image rotations at 90°, 180°, and 270° — all achieving ~125x GPU speedup due to memory-access-only computation

8. Conclusions and Future Work

8.1 Conclusions

This project successfully demonstrated that CUDA GPU parallelism delivers transformative speedups for image processing tasks. The key findings are:

1. All seven tested operations — Gaussian Blur, Sobel Edge Detection, Sharpening, Rotation ($90^\circ/180^\circ/270^\circ$), and 2D Convolution — exhibited substantial GPU speedups, ranging from 80x to 128x over equivalent CPU implementations on a 1024x1024 image.
2. Rotation operations achieved the highest speedups ($\sim 125x$) because they involve only index remapping with no floating-point arithmetic, making them purely memory-bandwidth limited and maximally exploitable by GPU parallelism.
3. Convolution-based operations (blur, edge detection, sharpen, generic 2D conv) consistently achieved $\sim 80x$ speedup, reflecting the combined benefit of thread-level pixel parallelism and independent per-pixel computation.
4. The scalability analysis confirmed that GPU advantage grows with image resolution — the speedup increased from $\sim 20x$ at 256x256 to $\sim 80x$ at 1024x1024 — indicating that GPUs become increasingly cost-effective as image dimensions scale.
5. The 16x16 thread block design (256 threads/block) proved highly effective, providing sufficient occupancy for the GPU's streaming multiprocessors while keeping boundary wastage minimal.

8.2 Future Work

- Shared Memory Optimization: Implement tiled convolution using CUDA shared memory to reduce redundant global memory reads per kernel, potentially improving convolution speedups by an additional 2-4x.
- Larger Kernels: Extend the generic convolution kernel to support 5x5 and 7x7 filters (e.g., large Gaussian blur, bilateral filter approximation) and study how speedup scales with kernel size.
- Color Image Processing: Extend all kernels to operate on 3-channel RGB images, processing channels independently or jointly, and measure performance impact.
- Multi-GPU and CUDA Streams: Implement pipeline parallelism using CUDA Streams to overlap data transfer and kernel execution, and explore multi-GPU partitioning for ultra-high-resolution images (4K, 8K).
- Real-Time Video Processing: Integrate the CUDA kernels into a video capture pipeline to process live webcam frames, validating the latency reduction in a real-time setting.
- Comparison with cuDNN/NPP: Benchmark custom kernels against NVIDIA's cuDNN and NPP (NVIDIA Performance Primitives) libraries, which use hand-tuned implementations, to assess the gap between research-grade and production-optimized code.

9. References

[1] J. D. Owens, D. Luebke, N. Govindaraju, M. Harris, J. Kruger, A. E. Lefohn, and T. J. Purcell, "A survey of general-purpose computation on graphics hardware," *Computer Graphics Forum*, vol. 26, no. 1, pp. 80–113, 2007.

- [2] P. Harish and P. J. Narayanan, "Accelerating large graph algorithms on the GPU using CUDA," in Proc. HiPC, 2007, pp. 197–208.
- [3] S. Ryoo, C. I. Rodrigues, S. S. Baghsorkhi, S. S. Stone, D. B. Kirk, and W.-M. W. Hwu, "Optimization principles and application performance evaluation of a multithreaded GPU using CUDA," in Proc. PPOPP, 2008, pp. 73–82.
- [4] H. Kaur and J. Singh, "Sobel edge detection using GPU," International Journal of Computer Applications, vol. 108, no. 14, pp. 28–31, 2014.
- [5] D. B. Kirk and W.-M. W. Hwu, Programming Massively Parallel Processors: A Hands-on Approach, 3rd ed. Morgan Kaufmann, 2016.
- [6] NVIDIA Corporation, CUDA C++ Programming Guide, Version 12.x, 2023. [Online]. Available: <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>
- [7] S. Barrett, "stb_image — Public Domain Image Loader," GitHub Repository, 2017. [Online]. Available: <https://github.com/nothings/stb>
- [8] R. C. Gonzalez and R. E. Woods, Digital Image Processing, 4th ed. Pearson, 2018.
- [9] A. Jain and R. Bhatt, "GPU accelerated image processing using CUDA," International Journal of Engineering Research and Applications, vol. 6, no. 3, pp. 42–47, 2016.
- [10] NVIDIA Corporation, "NVIDIA Performance Primitives (NPP)," 2023. [Online]. Available: <https://developer.nvidia.com/npp>